

RPAL Interpreter Project Report

Student Information

- Student 1 Name: TODO
- Student 1 ID: TODO
- Student 2 Name: TODO
- Student 2 ID: TODO

Project Overview

This project implements an RPAL interpreter in C++ without using ``lex``, ``yacc``, or parser generators. The executable is named ``rpal20`` and runs as:

```
```bash
./rpal20 file_name
```

The implementation tokenizes RPAL source, builds the raw AST using a hand-written recursive descent parser, standardizes the tree, and evaluates the standardized tree with a CSE-style runtime using lexical environments and closures.

## ## Program Structure

- ``Token.h``: token category and source-position data.
- ``lexical.h`` / ``lexical.cpp``: reusable scanner through ``tokenize(fileName)``.
- ``parser.h`` / ``parser.cpp``: recursive descent parser and AST stack builder.
- ``ASTNode.h`` / ``ASTNode.cpp``: AST node representation, printing, cloning, and cleanup.
- ``Standardizer.h`` / ``Standardizer.cpp``: AST-to-standardized-tree transformation.
- ``RuntimeValue.h`` / ``RuntimeValue.cpp``: runtime values and output formatting.
- ``Environment.h`` / ``Environment.cpp``: nested lexical environments.
- ``Builtins.h`` / ``Builtins.cpp``: RPAL built-in functions.
- ``CSEMachine.h`` / ``CSEMachine.cpp``: standardized tree evaluator.
- ``main.cpp``: command-line driver and debug flags.

## ## Main Function Prototypes

```
```cpp
vector<Token> tokenize(const string& fileName);
ASTNode* Parser::parse();
ASTNode* standardize(ASTNode* root);
RuntimeValue CSEMachine::evaluate(ASTNode* standardizedRoot);
void installBuiltins(Environment& env);
RuntimeValue applyBuiltin(const string& name,
                          const vector<RuntimeValue>& existingArgs,
                          const RuntimeValue& arg);
string formatRuntimeValue(const RuntimeValue& value);
```

Lexer Summary

The lexer follows ``RPAL_Lex.pdf``. It recognizes identifiers, integers, single-quoted strings, operator-symbol sequences, punctuation, and RPAL keywords. Spaces and ``//`` comments are deleted. Line and column values are stored only for diagnostics and are not printed during normal execution.

Parser Approach

The parser follows ``RPAL_Grammar.pdf`` with recursive descent functions for each

grammar nonterminal. Left-recursive grammar rules are implemented as loops, while right-associative rules such as ``**`` and ``->`` use recursive calls. AST nodes are created with the grammar labels required by the project, including ``lambda``, ``gamma``, ``tau``, ``within``, ``and``, ``rec``, ``fcn_form``, and operator labels.

AST Construction

The parser uses an AST stack. Leaf nodes are pushed when identifiers, integers, strings, and literals are consumed. Grammar reductions call ``buildTree(label, childCount)``, which pops children in reverse stack order and attaches them to a new parent node in left-to-right order.

Standardization Rules Implemented

The standardizer recursively standardizes every subtree and implements the core RPAL transformations:

- ``let X = E in P`` becomes ``gamma(lambda(X, P), E)``.
- ``P where X = E`` becomes ``gamma(lambda(X, P), E)``.
- ``X1 = E1 within X2 = E2`` becomes ``X2 = gamma(lambda(X1, E2), E1)``.
- ``rec X = E`` becomes ``X = gamma(ystar, lambda(X, E))``.
- ``F V1 ... Vn = E`` becomes nested unary lambdas.
- ``and`` definitions become comma/tau simultaneous bindings.
- Multi-parameter lambdas become nested unary lambdas.
- ``@ E1 N E2`` becomes ``gamma(gamma(N, E1), E2)``.

CSE Machine Design

The evaluator runs over the standardized tree using CSE-compatible runtime objects. Lambdas produce closures that store their defining environment, giving lexical scoping. Function application evaluates the rator and rand, then applies closures, built-ins, recursive ``ystar`` closures, or tuple selection. Conditional S evaluate only the selected branch.

Built-In Function Behavior

Implemented built-ins are ``Print``, ``Conc``, ``Stem``, ``Stern``, ``Order``, ``Null``, ``ItoS``, ``Isinteger``, ``Istruthvalue``, ``Isstring``, ``Istuple``, ``Isfunction``, and ``Isdummy``. ``Print`` writes the formatted argument followed by a newline and returns ``dummy``. Normal execution does not print returned values unless an RPAL program calls ``Print``.

Testing Method

The repository includes a repeatable diff-based workflow:

```
```bash
make test
```
```

The test script builds ``rpal20``, runs every file under ``tests/input``, writes outputs under ``tests/actual``, and compares them with ``tests/expected`` using ``diff -u``. If a reference executable is available, expected outputs can be regenerated with:

```
```bash
RPAL_REFERENCE=/path/to/rpal.exe ./run_tests.sh
```
```

Tests cover ``let``, precedence, curried functions, recursion, tuples, tuple

indexing, built-ins, and the sample program from the project specification.

Known Limitations

The implementation has been tested with the included suite, but this workspace does not contain `rpal.exe`, so exact reference verification must be run later with the official executable. Student names and IDs above must be filled in before final submission.